The Journal of Open Source Software

HydraR: Stateful Agentic Orchestration for Scientific Reproducibility in R

Chi Nam Ignatius Pang [1] **and Aidan Tay** [1]

1 Australian Proteome Analysis Facility (APAF), Macquarie University, Sydney, Australia

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

HydraR is a lightweight, state-managed framework designed to orchestrate complex "agentic" workflows—autonomous, multi-step processes driven by Large Language Models (LLMs) natively within R. While mainstream orchestration tools such as LangChain (Chase, 2022) and CrewAI (Moura, 2023) are predominantly Python-based, HydraR fulfills the R community's critical requirements for rigorous reproducibility, auditability, and seamless integration with established statistical pipelines.

Researchers can architect workflows as Directed Acyclic Graphs (DAGs) or iterative state machines, wherein each node represents an LLM-prompted task, a deterministic R function, or an autonomous auditor. HydraR ensures broad compatibility—supporting cloud APIs (Gemini (Google, 2025), Claude, OpenAI) and local models (Ollama)—while enforcing structural integrity through a built-in Advanced Validation Engine that performs deep static analysis of orchestration manifests before execution.

Statement of Need

The contemporary R ecosystem currently lacks a unified framework that provides both high-level agentic orchestration and robust, low-level filesystem safety. Existing packages, such as `ellmer` (Wickham et al., 2025), primarily focus on conversational interfaces, while `mall` (Couch, 2024) excels at mapping LLM inferences over textual data within data frames. Neither is structurally equipped to manage long-running, multi-agent collaborations that demand complex state persistence or isolated execution environments.

HydraR addresses three fundamental challenges in this domain: 1. **Auditability**: It implements a centralized `AgentState` with persistent checkpointing (via DuckDB (Raasveldt & Mühleisen, 2019)), yielding a complete, verifiable audit trail of all LLM interactions and state mutations. 2. **Filesystem Safety**: Leveraging Git Worktrees, HydraR isolates parallel agent executions into temporary branches, virtually eliminating data corruption and race conditions during concurrent operations. 3. **Reproducible Orchestration**: By defining workflows via `Mermaid.js` syntax and YAML manifests, HydraR decouples orchestration logic from the underlying execution framework. This declarative approach creates portable, shareable agentic protocols, effortlessly bypassing the added complexity typical when R is used to access multi-agent frameworks via `reticulate` (Ushey et al., 2024).

State of the Field

Although `reticulate` (Ushey et al., 2024) enables R users to interface with Python frameworks like LangChain, doing so introduces a structural impedance mismatch when serializing complex R data objects across language boundaries. HydraR avoids this by building natively on the R6

object-oriented system (Chang, 2021), preserving the fidelity of statistical objects while allowing direct compatibility with R's integrated debugging tool chains. Furthermore, unlike `ellmer` (Wickham et al., 2025) (which offers chat-centric workflows without DAG orchestration), and `gptstudio` (Nivard & Wade, 2024) (which serves as an IDE-centric coding assistant), `HydraR` is specifically engineered to manage the complete lifecycle of headless, parallel, multi-agent collaborations replete with robust state retention.

Software Design

`HydraR` employs a highly modular `R6` architecture spanning several key components: - `AgentLLMNode` / `AgentLogicNode`: Encapsulate LLM-driven prompts and deterministic pure-R logic steps, respectively. They utilize pluggable `AgentDriver` backends for provider-agnostic model communication. - `AgentDAG`: The core orchestration engine that manages node dependencies, validates network topology (via `igraph` (Csárdi & Nepusz, 2006)), and executes logic in parallel (via `furrr` (Vaughan & Reznik, 2022)). It features an integrated compiler-like Validation Engine that cross-references Mermaid topologies with `YAML` resource definitions and lints embedded R logic for compliance with safety standards (e.g., prohibiting imperative loops). - `AgentState`: A centralized state repository employing versioned history and custom reducers to systematically coordinate multi-agent outputs. - `WorktreeManager`: An engine that provisions isolated Git worktrees for parallel execution tasks, paired with an extensible `ConflictResolver` handling automated semantic resolution alongside human-in-the-loop task reconciliation.

Research Applications

Multimodal Travel Itinerary Planner

In this example, an LLM `Planner` agent generates a draft itinerary, which is then audited by a deterministic `Validator` against hard constraints. Successfully validated plans proceed to a *multimodal image generation* phase where **Gemini 3.1 Flash** (Google, 2025) creates visual assets based on the itinerary's locales. Finally, a logic node renders a bespoke, CSS-styled HTML pamphlet. Crucially, **conditional looping** between the planner and validator enforces an iterative refinement cycle until all strict parameters are met (section 5.1).

Page 1: Multi-modal travel pamphlet showcasing stateful orchestration and image generation.

Figure 1: Page 1: Multi-modal travel pamphlet showcasing stateful orchestration and image generation.

Page 2: Final formatted CSS-styled HTML itinerary.

Figure 2: Page 2: Final formatted CSS-styled HTML itinerary.

Generated Multi-modal travel pamphlet for a Sydney to Hong Kong itinerary.

Declarative Workflow Excerpt

```
graph: |
  graph LR
    Planner["Travel Planner | type=llm | role_id=travel_concierge"]
    Validator["Constraint Auditor | type=logic | logic_id=validate_constraints"]
    ImageGate["Image Gate | type=logic | logic_id=check_image_status"]
    ImageGenerator["Image Generator | type=logic | logic_id=generate_and_save_images"]
    TemplateManager["Template Provider | type=logic | logic_id=provide_template"]
```

DRAFT

```
    PamphletFormatter["Pamphlet Formatter | type=logic | logic_id=format_pamphlet"]
    Finalizer["Itinerary Saver | type=logic | logic_id=save_itinerary"]

    Planner --> Validator
    Validator -- "fail" --> Planner
    Validator -- "pass" --> ImageGate
    ImageGate --> ImageGenerator
    ImageGate --> TemplateManager
    ImageGenerator --> TemplateManager
    TemplateManager --> PamphletFormatter
    PamphletFormatter --> Finalizer

roles:
  travel_concierge: >
    You are a professional travel concierge...
```

Visual DAG structure of the multi-modal travel itinerary planner, featuring iterative validation loops and parallel generation paths.

Figure 3: Visual DAG structure of the multi-modal travel itinerary planner, featuring iterative validation loops and parallel generation paths.

R Orchestration (Minimal)

```r
library(HydraR)
wf <- load_workflow("hong_kong_travel.yml")
dag <- spawn_dag(wf, auto_node_factory())
results <- dag$run(initial_state = wf$initial_state)
```

Parallel Sorting Algorithm Comparison

This benchmarking example tasks three distinct LLM agents with simultaneously implementing different sorting algorithms. `HydraR` executes each task within an isolated `Git worktree` to aggressively uncouple filesystem side-effects and prevent merge conflicts. Following execution, a Merge Harmonizer noded systematically reconciles the independent branches back to the main state, preceding a terminal logic node that empirically benchmarks the aggregated algorithms across five continuous trials (Figure 4).

Sorting Algorithm Performance Benchmark (1,000 elements over 5 trials).

Figure 4: Sorting Algorithm Performance Benchmark (1,000 elements over 5 trials).

Declarative Workflow Excerpt

```
graph: |
  graph LR
      bubble["Bubble Agent | type=llm | role_id=bubble"]
      quick["Quick Agent | type=llm | role_id=quick"]
      merge["Merge Agent | type=llm | role_id=merge"]
      merger["Merge Harmonizer | type=merge"]
      benchmark["Benchmark | type=logic | logic_id=run_benchmark"]

      bubble --> merger
      quick --> merger
      merge --> merger
      merger --> benchmark
```

Visual representation of the parallel sorting comparison workflow, showing the fan-out/fan-in pattern and the merge harmonizer.

Figure 5: Visual representation of the parallel sorting comparison workflow, showing the fan-out/fan-in pattern and the merge harmonizer.

R Orchestration (Parallel with Isolation)

```r
library(HydraR)
wf <- load_workflow("sorting_benchmark.yml")
dag <- spawn_dag(wf, auto_node_factory())
results <- dag$run(initial_state = wf$initial_state, use_worktrees = TRUE)
```

Fault-Tolerant Pipelines with DuckDB Persistence

To mitigate transient failures inherent to long-running scientific workflows, `HydraR` integrates advanced checkpoint-and-resume functionality powered by DuckDB (Raasveldt & Mühleisen, 2019). Whenever a pipeline pauses mid-execution, it automatically persists its complete `AgentState`. Upon restart, any operator-applied programmatic interventions are seamlessly merged, actively preventing redundant evaluations by allowing execution to jump directly back to the paused node.

Declarative Workflow Definition

```yaml
graph: |
  graph LR
    Step1["Initialization"] --> Step2["Risky Logic"]
    Step2["Risky Logic"] --> Step3["Conclusion"]
logic:
  check_fixed: |
    if (!isTRUE(state$get("fixed")))
      return(list(status = "PAUSE", output = "Waiting for manual fix."))
    list(status = "SUCCESS", output = "System recovered!")
```

Visual logic of the fault-tolerant pipeline with DuckDB persistence, illustrating a linear flow with restartable checkpoints.

Figure 6: Visual logic of the fault-tolerant pipeline with DuckDB persistence, illustrating a linear flow with restartable checkpoints.

Checkpoint and Resume

```r
library(HydraR)
wf    <- load_workflow("state_persistence.yml")
dag   <- spawn_dag(wf)
saver <- DuckDBSaver$new(db_path = "history.duckdb")
tid   <- "reprex-session-001"

# Run 1: Pauses at Step2 — state saved to DuckDB
res1 <- dag$run(thread_id = tid, checkpointer = saver,
                initial_state = list(fixed = FALSE))

# Run 2: Fix applied, resume from Step2
res2 <- dag$run(thread_id = tid, checkpointer = saver,
                initial_state = list(fixed = TRUE),
                resume_from = "Step2")
```

DRAFT

By safeguarding computational progress autonomously, this pattern effectively circumvents prohibitively costly re-execution in data science pipelines reliant on voluminous LLM invocations or laborious large-scale database queries.

Research Impact Statement

`HydraR` is actively deployed at the **Australian Proteome Analysis Facility (APAF)** to architect and automate resilient bioinformatics tooling. Its declarative YAML schemas empower non-programming domain experts to routinely govern advanced LLM logic pipelines without any R coding proficiency, thereby democratizing the scale and reproducibility of model-assisted scientific research.

AI Usage Disclosure

Transparency and accountability are core tenets of the `HydraR` paradigm. A substantial portion of the implementation was co-developed utilizing **Antigravity**, an agentic AI coding assistant, supervised by a rigorous "human-in-the-loop" review protocol. Specifically, the original authors independently conceptualized the underlying system architecture, while the AI generated specialized logic, unit tests, and software documentation with every resultant commit sequentially tested and manually verified.

Acknowledgements

We acknowledge funding from **Bioplatforms Australia**, enabled by the **National Collaborative Research Infrastructure Strategy (NCRIS)**.

References

Chang, W. (2021). *R6: Recursive object-oriented class system*. https://R6.r-lib.org

Chase, H. (2022). LangChain: Building applications with LLMs through composability. *GitHub Repository*. https://github.com/langchain-ai/langchain

Couch, S. P. (2024). *Mall: Run language models multiple times*. https://github.com/simonpcouch/mall

Csárdi, G., & Nepusz, T. (2006). The igraph software package for complex network research. *InterJournal*, *Complex Systems*, 1695. https://igraph.org

Google. (2025). *Gemini 1.5 flash*. https://gemini.google.com.

Moura, J. (2023). CrewAI: Collaborative AI agents. *GitHub Repository*. https://github.com/joaomoura/crewai

Nivard, M., & Wade, J. (2024). *Gptstudio: Use large language models directly in your development environment*. https://github.com/MichelNivard/gptstudio

Raasveldt, M., & Mühleisen, H. (2019). DuckDB: An embeddable analytical database. *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*. https://doi.org/10.1145/3299869.3320212

Ushey, K., Allaire, J., & Tang, Y. (2024). *Reticulate: Interface to 'python'*. https://rstudio.github.io/reticulate/

Vaughan, D., & Reznik, D. (2022). *Furrr: Apply mapping functions in parallel using futures*. https://github.com/DavisVaughan/furrr

DRAFT

Wickham, H., Cheng, J., Jacobs, A., Aden-Buie, G., & Schloerke, B. (2025). *Ellmer: Chat with large language models*. https://ellmer.tidyverse.org

DRAFT